

REDEFLIX (Sistema de Streaming Adaptativo (DASH))

Este projeto é um sistema cliente-servidor focado na transmissão eficiente de vídeo sob demanda, utilizando a especificação **DASH (Dynamic Adaptive Streaming over HTTP)**.

Em vez de transferir um arquivo de mídia monolítico, a aplicação fragmenta o vídeo em pequenos segmentos temporais (*chunks*) disponíveis em múltiplas resoluções. O front-end, operando de forma distribuída na rede local, atua como um reprodutor autônomo que gerencia o download e a injeção progressiva dos fluxos de áudio e vídeo utilizando a API nativa de **Media Source Extensions (MSE)**.

Principais Pilares do Sistema:

- **Adaptação Dinâmica (ABR):** Algoritmo que calcula o desempenho da rede em tempo real e chaveia a qualidade do vídeo sob demanda, sem causar travamentos na reprodução.
- **Gestão de Memória (Janela Deslizante):** Um *Garbage Collector* customizado que monitora a tabela de páginas da memória e remove ativamente blocos de vídeo já assistidos para evitar *Memory Leaks* no navegador.
- **Orquestração de Concorrência:** Implementação de travas (*locks*) assíncronas baseadas em eventos de I/O para evitar Condições de Corrida (*Race Conditions*) durante operações paralelas de rede e manipulação do *SourceBuffer*.
- **Descoberta de Nós Agnostic:** Uso do contexto de execução (`window.location.hostname`) para inferir a topologia da rede sem depender de IPs fixos no código cliente.

Tecnologias Utilizadas:

Front-end e Reprodutor

- Vue.js 3 (Composition API)
- Vite (Build Tool e Servidor de Desenvolvimento)
- HTML5 Media Source Extensions (MSE) para manipulação de buffer
- TailwindCSS e Bootstrap para estilização da interface

Back-end e Infraestrutura

- Nginx (Servidor Web para distribuição de mídia estática e configuração de CORS)
- Go (API REST para consumo dinâmico do catálogo de vídeos)
- Docker e Docker Compose (Orquestração, portabilidade e isolamento dos serviços)
- FFmpeg (Transcodificação e fragmentação dos vídeos para o padrão MPEG-DASH)

Como Executar:

Requisitos

- **Docker** e **Docker Compose** instalados na sua máquina.

Nota: A aplicação foi arquitetada utilizando o padrão de microsserviços e está 100% containerizada. Não é necessário ter Node.js, Go, Nginx ou qualquer outra dependência instalada localmente no seu sistema operacional. O Docker orquestrará todo o ecossistema distribuído.

Instruções de Execução

1. Clone o repositório:

```
git clone https://github.com/yurimutz/Trabalho-Redes
cd Trabalho-Redes
```

2. Baixe os vídeos no google drive e coloque em alguma pasta /backend-go/meus_videos_gerados/

Baixe alguma pasta no nosso [drive](#) como exemplo de vídeo e adicione em umas das pastas /meus_videos_gerados/ do Servidor.

3. Suba a infraestrutura completa:

```
docker-compose up -d --build
```

4. Acesse e teste a aplicação:

- **Acesso Local (na mesma máquina):** Abra o seu navegador e acesse <http://localhost:3000>.
- **Acesso Distribuído (em outro dispositivo da rede):** Descubra o IP local do computador onde o Docker está rodando (ex: [192.168.0.15](#)). Pegue o seu celular ou outro PC conectado ao mesmo Wi-Fi e acesse <http://192.168.0.15:3000>. O cliente fará a descoberta dinâmica do IP e buscar os vídeos no Nginx perfeitamente.

Funcionalidades Implementadas:

- **Catálogo Dinâmico Integrado:** Consumo assíncrono de API REST para renderização do catálogo de mídias disponíveis.
- **Reprodutor DASH Nativo (MSE):** Decodificação manual e injeção progressiva de fragmentos separados de vídeo ([.m4s](#)) e áudio diretamente em instâncias de [SourceBuffer](#) utilizando a API Media Source Extensions.
- **Adaptive Bitrate (ABR) Empírico:** Algoritmo que calcula o *throughput* real da rede a cada download de chunk (cruzando tamanho binário com [performance.now\(\)](#)) e altera a resolução do vídeo dinamicamente com base em uma média móvel.
- **Janela Deslizante de Memória (Sliding Window):** Um coletor de lixo (Garbage Collector) customizado que monitora a tabela de páginas da memória e remove ativamente blocos de vídeo já assistidos (mais de 30 segundos no passado) para prevenir *Memory Leaks* e travamentos.
- **Controle de Concorrência e Travas Assíncronas:** Implementação de semáforos lógicos (Locks) atrelados ao evento de I/O [updateend](#) para coordenar o Event Loop e impedir Condições de Corrida (*Race Conditions*) na manipulação da RAM.
- **Descoberta Dinâmica de Nós (Zero-Config LAN):** Resolução de topologia de rede em tempo de execução via [window.location.hostname](#), viabilizando o uso do sistema distribuído por qualquer dispositivo da rede sem *hardcoding* de IPs.
- **Sincronização de Seeking Completa:** Interceptação avançada de cliques na barra de progresso do player com recálculo matemático seguro do índice dos chunks temporais e reestruturação do buffer.

Possíveis Melhorias Futuras:

- **Migração de Armazenamento para Nuvem (AWS):** Desacoplar os arquivos de mídia da infraestrutura local, hospedando os manifestos e *chunks* `.m4s` em *buckets* do **Amazon S3** distribuídos via **Amazon CloudFront** (CDN Edge). Essa abordagem removeria a carga de I/O dos contêineres Nginx/Go, garantindo alta disponibilidade, escalabilidade elástica sob demanda e menor latência na entrega dos segmentos de vídeo para usuários em diferentes áreas geográficas.
- **Algoritmos ABR Preditivos:** Substituir a atual média móvel empírica por heurísticas avançadas de otimização (como o algoritmo BOLA - *Buffer Occupancy based Lyapunov Algorithm*), que ponderam tanto a oscilação da rede quanto o nível de preenchimento atual da RAM para tomar decisões mais suaves.
- **Cache em Camada Interceptadora (Service Workers):** Implementar um *Service Worker* para interceptar os `fetch` de rede e armazenar fragmentos já baixados no **Cache Storage** do navegador. Isso zeraria o custo de rede ao usuário realizar retrocessos (Seeks) na linha do tempo do vídeo.
- **Sincronização Multi-Cliente (Watch Party):** Integração do reprodutor com um servidor de WebSockets para estabelecer comunicação bidirecional de baixa latência, permitindo que a reprodução, pausas e pulos no vídeo sejam sincronizados simultaneamente entre várias máquinas da rede local.
- **Proteção Criptográfica (DRM):** Utilização da API nativa *Encrypted Media Extensions (EME)* em conjunto com o player para descriptografar segmentos de vídeo em tempo real no buffer, garantindo segurança na distribuição da mídia contra cópias não autorizadas.

Documentação Técnica do Projeto de Streaming (MPEG-DASH)

[Documento técnico completo](#) que detalha toda a infraestrutura, decisões de engenharia de software, modelos matemáticos e otimizações implementadas no ecossistema (Front-end em Vue.js, Nginx, Docker, Go e FFmpeg).

Principais Tópicos Abordados no Documento

1. **Orquestração e Topologia de Rede:** Detalhes sobre a arquitetura de microsserviços containerizados em Docker, o padrão *Backend For Frontend* (BFF) com agregador e a camada de *Edge Cache* com Nginx atuando como uma CDN interna para aliviar gargalos de I/O.
2. **Gerenciamento Estrito de Memória (Sliding Window):** Explicação da janela deslizante assíncrona, detalhando o comportamento do buffer futuro (limite de 40 segundos como *Bounded Buffer*) e do *Garbage Collector* agressivo no passado (limite de 10 segundos) para evitar vazamentos de memória (*Memory Leaks*).
3. **Algoritmo de Adaptive Bitrate (ABR) Avançado:** Heurística empírica de transição de qualidade utilizando média ponderada (pesos 5-3-2), o mecanismo de janela deslizante real com fila rotativa contínua (`push` e `shift`) e a **Trava de Inércia Direcional (Cooldown)** com lógica assimétrica para mitigar o Efeito Ping-Pong.
4. **Concorrência e Segurança de I/O:** Implementação de travas lógicas (semáforos assíncronos) baseadas no evento nativo `updateend` da API *Media Source Extensions* (MSE) para blindar o sistema contra Condições de Corrida (*Race Conditions*) durante o *seeking*.
5. **Processamento Offline e Transcodificação (FFmpeg):** O comando completo de transcodificação com múltiplas saídas dissecado bloco a bloco, detalhando os filtros de imagem (`scale`, `pad`, `setsar`), a

sincronia perfeita de fragmentos (`force_key_frames`) e a estruturação da *Bitrate Ladder* (200k, 800k, 3000k, 6000k).